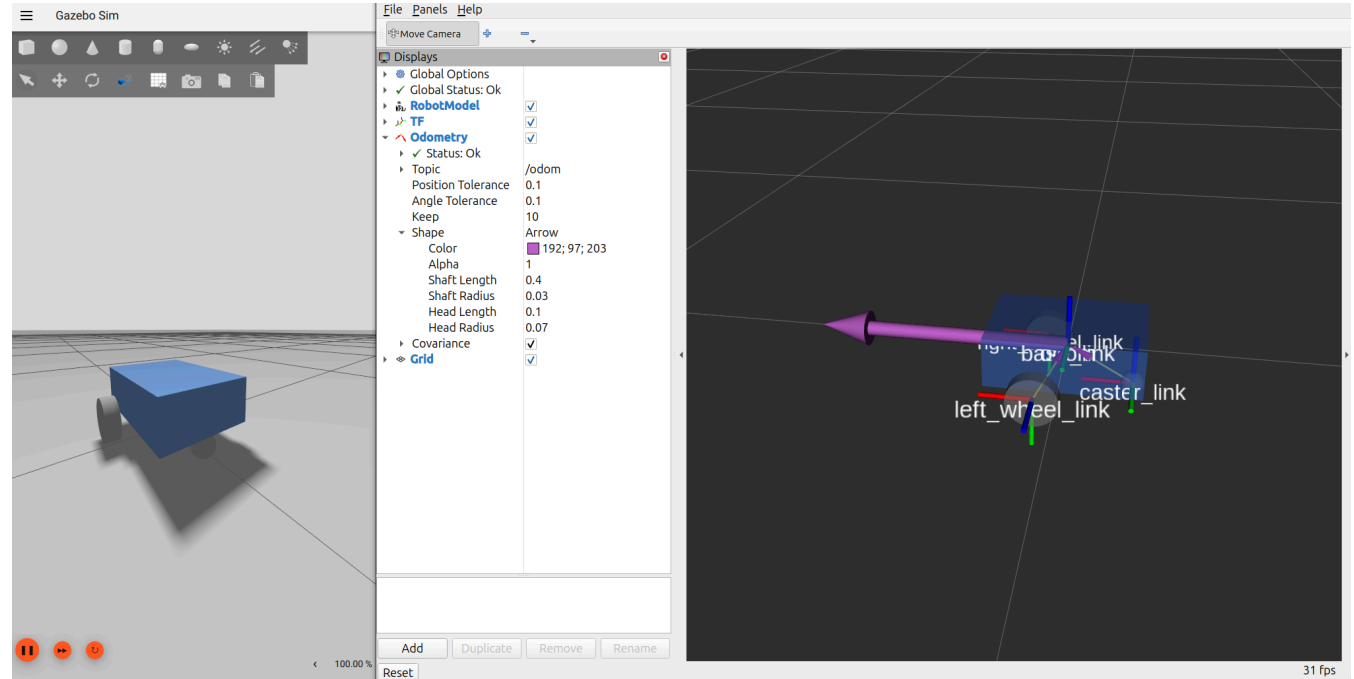
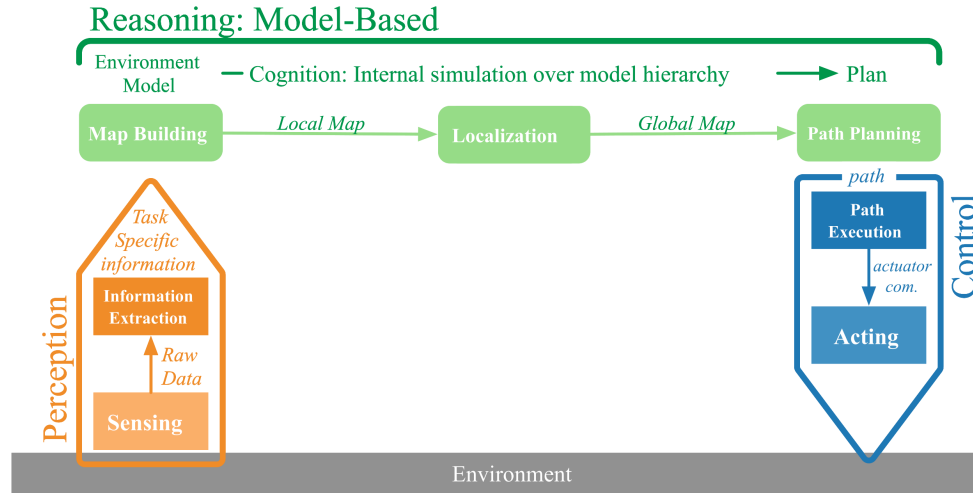


Autonomous Systems and Mobile Robots

Robot Body and Blind Locomotion



Perceive · Reason · Act



Robot-centric geometry

How is each link positioned relative to the chassis?

Proprioceptive estimation

How does a robot estimate its displacement without external reference?

Semester at a Glance

#	Date	Theme
0	14.04	Python Intro
1	21.04	ROS2 Packaging + PID Control
2	05.05	Robot Body (Proprioception) and Blind Locomotion ← you are here
3	12.05	Exteroception — LiDAR
4	19.05	MiniLab 1 Briefing
—	26.05	<i>Pfingsten — no session</i>
5	02.06	Troubleshooting (<i>tbd</i>)
6	09.06	FK/IK Robotic Manipulator
7	16.06	MiniLab 2 Briefing
8	23.06	Troubleshooting (<i>tbd</i>)
9	30.06	Recap (Minilab and Tutorial)

Last Week

ROS 2 Communication Patterns

- Package structure and `colcon` build
- Publisher / subscriber nodes
- Topics, Services and Actions
- Launch files and parameters

PID Control

- `G01` package — robot control
- JupyterNotebook `Car Example`





Quick Check-in

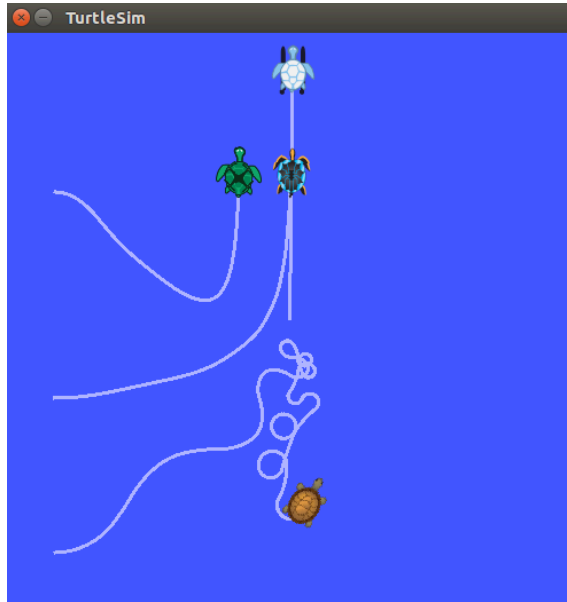
Are you keeping up? Any issues with the exercises? General concerns?

Takes 5 minutes — anonymous.

docs.google.com/forms/d/e/1FAIpQLSehgajKxiOdVcmqd08l-6CsvgbcNR3ModirBnMyRwbh8dqV9Q/viwwform

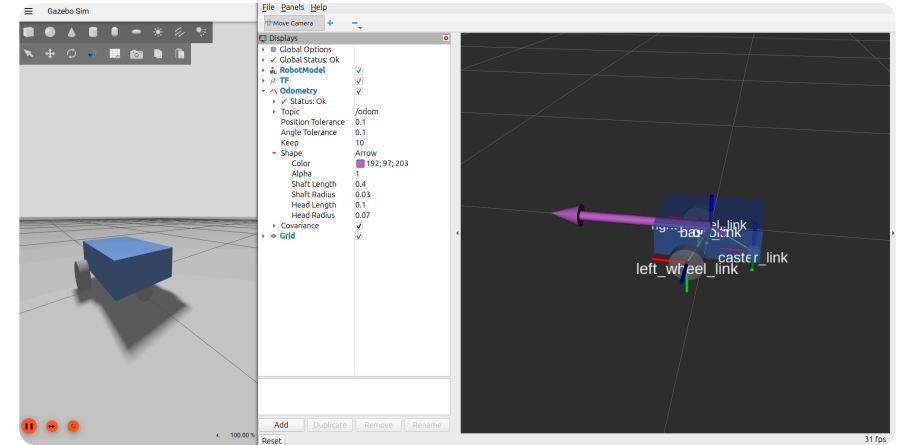
The Platform

WHAT YOU HAVE SEEN



Simulated turtle · no real body · no physics

WHAT YOU WILL BUILD TODAY



Custom URDF · from scratch · yours to extend

Today's content

The Robot Body

Coordinate frames and REP 103

Homogeneous transforms

URDF — links and joints

Kinematic chain and TF tree

Simulation

Gazebo plugins

ROS–Gazebo bridge

Blind Locomotion

Odometry — `odom → base_link`

Dead-reckoning and drift

The Robot Body

"How is each link positioned relative to the chassis?"

Coordinate frames

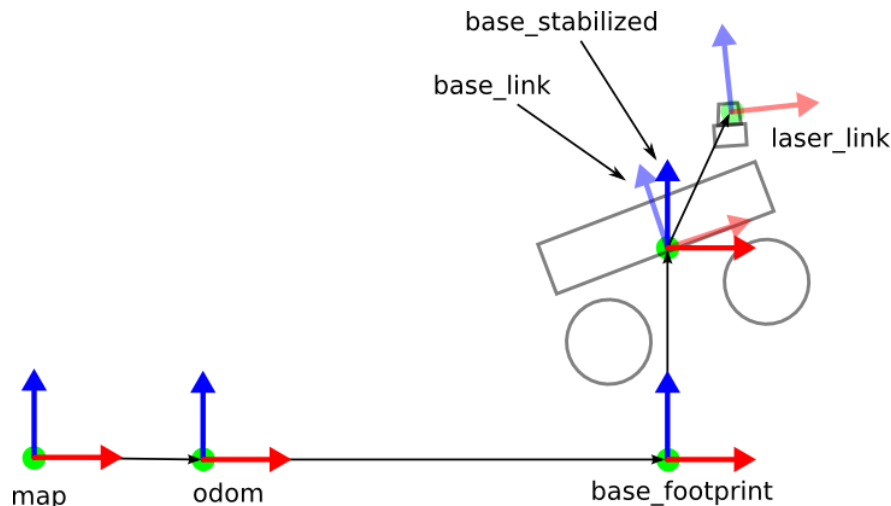
Why frames?

A LiDAR reports an obstacle at 0.8 m straight ahead — but straight ahead *of what?* The sensor? The chassis? The map?

Every measurement and every command lives in a **frame**. A frame is a named origin with three axes attached to some part of the robot or the world.

A transform between two frames answers: "*where is B, seen from A?*"

`tf2` is the ROS 2 library that stores and queries all transforms in the system — at any point in time. You describe the robot; `tf2` does the bookkeeping.



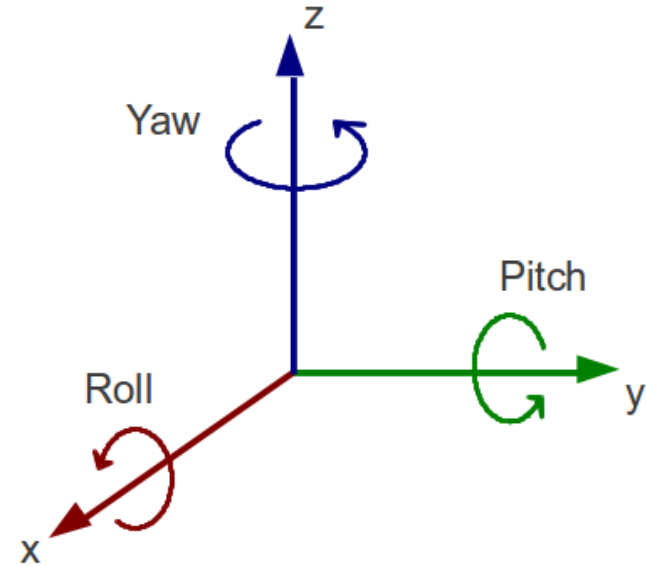
Frame tree on a real robot

Coordinate Conventions

REP 103 - Right Handed Coordinate System

x forward · y left · z up

Rotation	Axis
Roll	x
Pitch	y
Yaw	z



REP 103 — x forward, y left, z up

Homogeneous Transforms

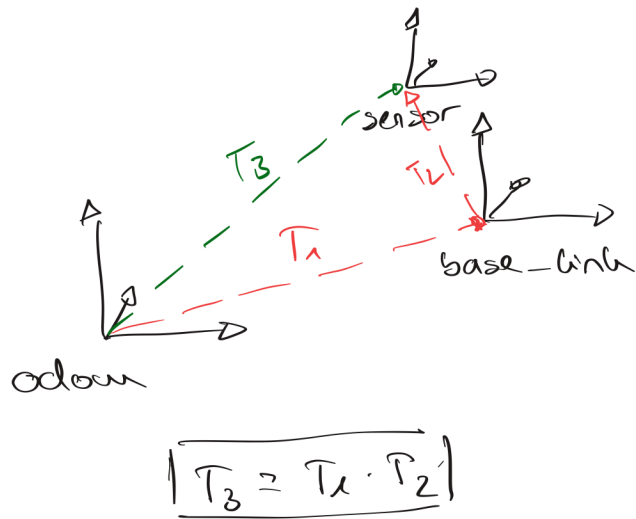
A frame relationship is a matrix

A homogeneous transformation matrix encodes translation and rotation in one object.

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \quad R \in \mathbb{R}^{3 \times 3}, \mathbf{t} \in \mathbb{R}^3$$

- R — rotation derived from the `rpy` angles
- $\mathbf{t}$ — translation from the `xyz` offset

Chaining two frames is matrix multiplication



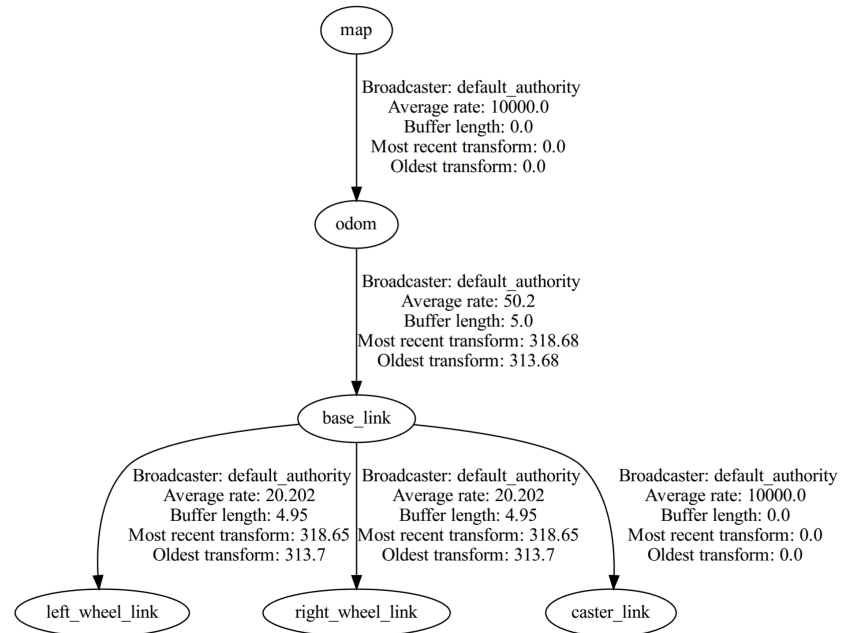
Chaining transforms via matrix multiplication

Coordinate Frames

Every joint defines a frame

A link's position in the robot is specified by its **joint origin** — a translation and rotation relative to the parent frame.

- Translation (`xyz`): offset from parent frame origin
- Rotation (`rpy`): reorients the child frame axes



Joint origin in parent frame

URDF — Links

Links — rigid bodies in the kinematic tree

A `<link>` element declares a rigid body with three sub-elements:

- `<visual>` — rendered geometry and material
- `<collision>` — simplified shape for contact physics
- `<inertial>` — mass and moment of inertia

All three are required for a physically plausible simulation.

```
14 <!-- ===== Chassis ===== -->
15 <!-- Box: 0.30 m (x) x 0.20 m (y) x 0.10 m (z), mass 5 kg -->
16 <!-- Inertia: Ixx=m(y²+z²)/12 Iyy=m(x²+z²)/12 Izz=m(x²+y²)/12 -->
17 <link name="base_link">
18   <visual>
19     <geometry>
20       <box size="0.30 0.20 0.10"/>
21     </geometry>
22     <material name="blue"/>
23   </visual>
24   <collision>
25     <geometry>
26       <box size="0.30 0.20 0.10"/>
27     </geometry>
28   </collision>
29   <inertial>
30     <mass value="5.0"/>
31     <inertia ixx="0.02083" ixy="0.0" ixz="0.0"
32               iyy="0.04167" iyz="0.0"
33               izz="0.05417"/>
34   </inertial>
35 </link>
```

URDF link element

URDF — Joints

The Connections between links

A `<joint>` element connects two links and defines what motion is permitted:

- `type` — motion constraint
- `origin` — pose of child frame in parent frame
- `axis` — rotation axis (non-fixed joints)

Joint types used this week:

- `continuous` — unbounded rotation (drive wheels)
- `fixed` — no motion (caster)

The sequence of links connected through joints is the robot's kinematic chain.

```
37  <!-- ===== Left wheel ===== -->
38  <!-- Cylinder: r=0.05 m, l=0.03 m, mass 0.5 kg -->
39  <!-- Joint rotated -π/2 around X so cylinder axis (local Z) → world Y -->
40  <!-- Inertia: Ixx=Iyy=m(3r²+l²)/12  Izz=mr²/2 -->
41  <joint name="left_wheel_joint" type="continuous">
42    <parent link="base link"/>
43    <child link="left_wheel_link"/>
44    <origin xyz="0.05 0.13 -0.05" rpy="-1.5708 0.0 0.0"/>
45    <axis xyz="0 0 1"/>
46    <dynamics damping="0.1" friction="0.1"/>
47  </joint>
48
49  <link name="left_wheel_link">
50    <visual>
51      <geometry>
52        <cylinder radius="0.05" length="0.03"/>
53      </geometry>
54      <material name="grey"/>
55    </visual>
56    <collision>
57      <geometry>
58        <cylinder radius="0.05" length="0.03"/>
59      </geometry>
60    </collision>
61    <inertial>
62      <mass value="0.5"/>
63      <inertia ixx="0.00035" ixy="0.0" ixz="0.0"
64                iyy="0.00035" iyz="0.0"
65                izz="0.000625"/>
```

URDF joint element

From static description to live transforms

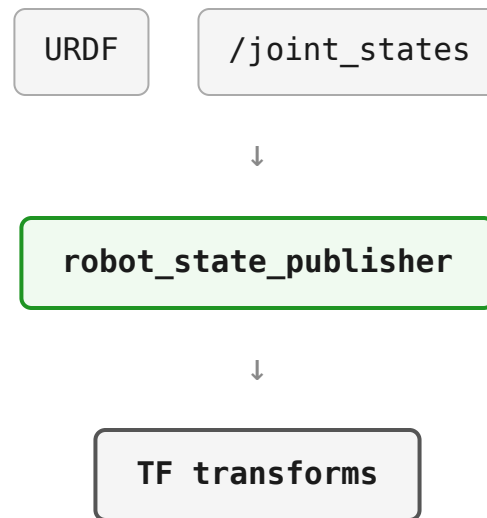
`robot_state_publisher`

The URDF contains geometry — it says nothing about the robot's current pose.

`robot_state_publisher` bridges the static description and the runtime state. It reads two inputs:

- The URDF — loaded once at startup
- `/joint_states` — joint angles published continuously at runtime

For every joint it computes and broadcasts a TF transform. Static joints are published once; moving joints are updated on every `/joint_states` message.



Local Motion Estimate

"How does a robot estimate its displacement without external reference?"

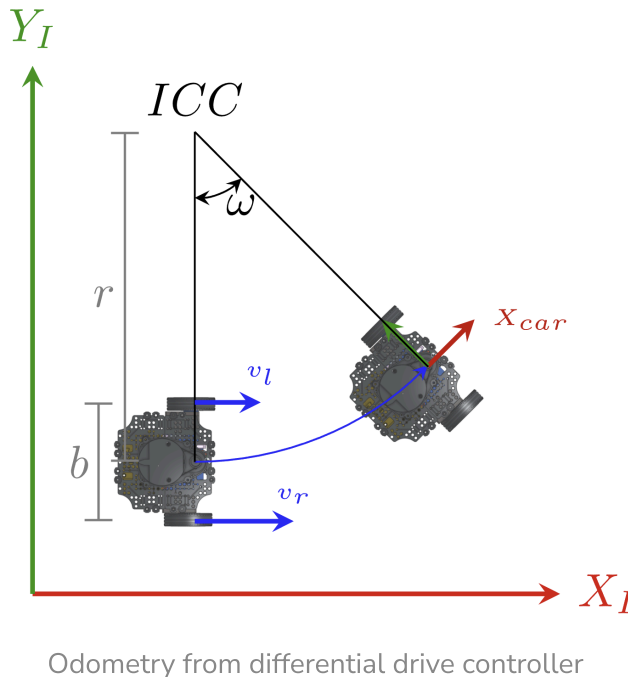
Dead-reckoning - position from motion alone

Dead-reckoning estimates position by integrating motion commands over time — using only internal information, no external reference.

In ROS 2 this appears as the `odom → base_link` transform:

- `odom` — fixed at the robot's starting position, does not move
- `base_link` — rigidly attached to the chassis, moves with the robot

The transform between them is the robot's running estimate of where it is.

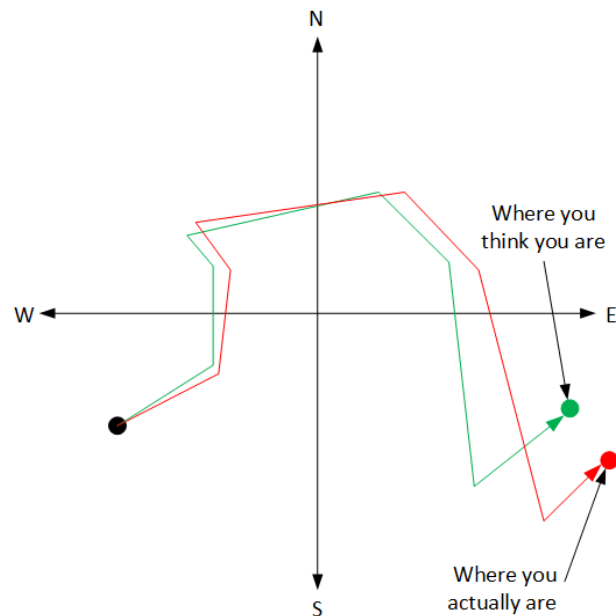


Error Accumulation in Dead-reckoning

Each integration step introduces a small error. Over time these errors accumulate — the estimate diverges from ground truth:

- **Wheel slip** — contact force insufficient to transfer commanded motion
- **Timing imprecision** — commands held for slightly incorrect durations
- **Surface irregularities** — unexpected resistance or disturbance

Errors are individually small but **accumulate monotonically** — the estimate diverges from ground truth over time.



Commanded vs. actual path

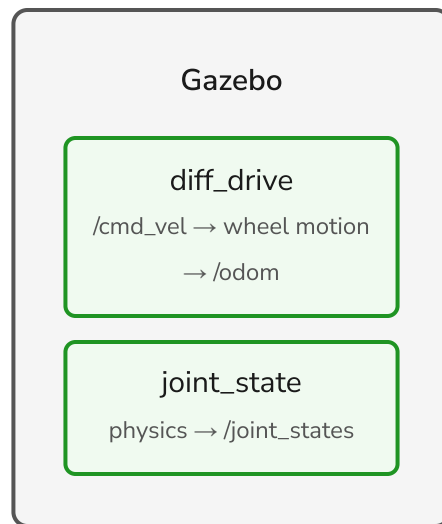
Simulation with Gazebo

RViz visualises the model — Gazebo makes it move

Plugins

Gazebo's physics engine knows nothing about ROS.
Plugins are the pieces of code that connect Gazebo's internals to ROS topics.

Plugin	Behaviour
Differential drive	Receives <code>/cmd_vel</code> , drives the wheels, runs the dead-reckoning process and publishes the <code>odom</code> → <code>base_link</code> transform
Joint state publisher	Reports actual joint angles from the physics engine as <code>/joint_states</code>

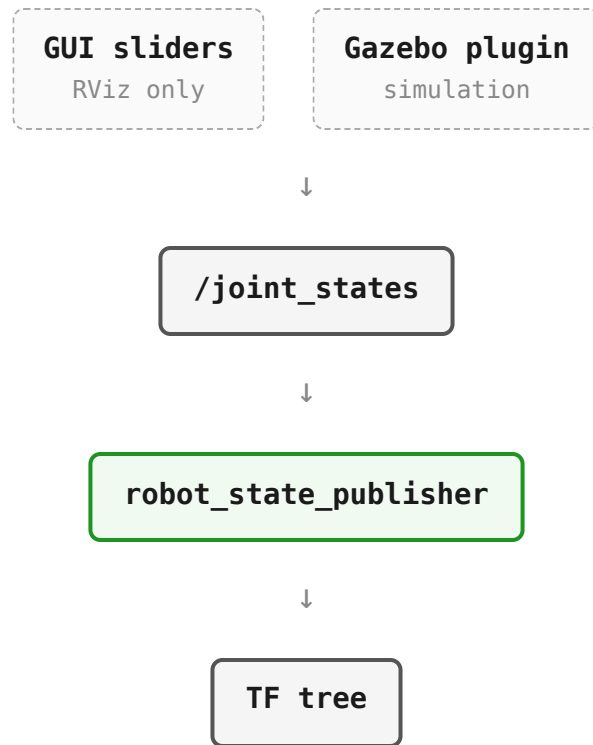


Source of Joint States

`robot_state_publisher` is always the same — the source of `/joint_states` changes:

- **RViz only** — GUI sliders
(`joint_state_publisher_gui`) let you pose the robot manually
- **Gazebo** — the joint state plugin reads angles from the physics engine

The `/joint_states` interface is identical in both cases; `robot_state_publisher` does not care which side produces it.



The Bridge

Separate middleware, explicit translation

Gazebo and ROS 2 use incompatible pub-sub systems — they cannot see each other's topics.

`ros_gz_bridge` connects them by mapping specific topics explicitly:

- Each entry declares a **topic name**, **message type**, and **direction**
- Only listed topics are bridged — everything else stays isolated

Topic	Direction	Purpose
<code>/cmd_vel</code>	ROS → GZ	Drive the robot
<code>/odom</code>	GZ → ROS	Dead-reckoning output from diff_drive
<code>/joint_states</code>	GZ → ROS	Feed <code>robot_state_publisher</code>
<code>/clock</code>	GZ → ROS	Sync simulation time to ROS nodes

What You Will Build

Your ROS 2 workspace

my_robot_description	model	URDF · geometry · inertia
my_robot_bringup	assembly	launch · world · bridge · RViz config
my_robot_control	behaviour	movement · trajectories

Reference solution

